AN0064
Application Note

AT32 Motor Control Library User Guide

Introduction

This application note describes how to use AT32 motor control library in detail with example cases, including the setup of the hardware and software environments, motor control library architecture, header file settings, and rich choices of functions.

Applicable products:

Part number	AT32F series AT32L series AT32M series

Contents

List of tables

List of figures

1 Overview

Motor control library algorithms are as follows:
Target motor type: Three-phase permanent magnet synchronous motor (BLDC)
Control methods:

- FOC vector control
- 120° square wave commutation

Three-phase PWM method:

- SVPWM
- 120° conduction PWM control

Phase current sensing methods:

- 3-shunt current sensing
- 2-shunt current sensing
- 1-shunt current sensing and reconstruction method

Rotor position detection modes:

- Hall-effect position sensor
- Photoelectric incremental encoder
- Absolute Magnetic Encoder

Initial rotor position estimation modes:

- Three-phase voltage vector mode
- Two-phase voltage vector mode

Rotor position estimation in FOC driving modes:

- BEMF estimation with Luenberger observer

Rotor position estimation in 120° square wave control mode:

- BEMF zero crossing point detection by comparator
- BEMF zero crossing point detection by ADC

Sensored FOC sine-wave control mode:

- Voltage vector control
- Torque control (current vector control)

- Speed control
- Field weakening control
- Positioning control
- Regenerative braking control

Sensorless FOC sine-wave control mode:

- Open loop startup
- Torque control (current vector control)
- Speed control
- Field weakening control
- Regenerative braking control
- Upwind/Downwind startup

Sensored 120° square-wave commutation mode:

- 120° square wave voltage control
- Torque control (120° square wave current control)
- Speed control

Sensorless 120° square-wave commutation mode:

- 120° square wave voltage control
- Open loop startup
- Torque control (120° square-wave current control)
- Speed control

Auto tuning functions:

- Hall state sequence self-learning (for Hall sensor only)
- Motor winding parameters identification
- Current PI control parameters self-tuning
- Magnetic encoder deviation angle self-calibration

2 Environment requirements

2.1 Hardware environment

Hardware resources required include a PMSM (BLDC) motor, AT-Link or third-party debugger and a motor control board. If the AT-MOTOR-EVB evaluation board is to be used, please refer to the *AT32_LV_Motor_Control_EVB_User_Manual* for details on hardware configurations.

- PMSM (BLDC) motor
- Debugger
- Motor control board

2.2 Software environment

1. Set up an AT32 development environment according to the getting-started guidelines of the corresponding AT32 MCU series and their respective functions.
2. After a new project is created, add the header files and functions of the motor control library to the new project, and configure motor control mode, drive parameters, control board parameters, controller parameters and MCU peripheral parameters in the header files.
3. Keil IDE environment, it is necessary to modify “Read/Only Memory” in “Options” depending on AT32 MCU Flash memory size. See Table 1 for details.
   Example, for a 256-KB AT32F413RCT7, its IROM1 start address is at 0x8000000 with a size of 0x3F800, while its IROM2 start address is at 0x803F800 with a size of 0x800, as shown in Figure 1.
   For AT32IDE environment, it is necessary to modify ID file according to AT32 MCU Flash memory size, as shown in Figure 2.
   For IAR Systems, it is necessary to modify *icf file according to AT32 MCU Flash memory size, as shown in Figure 3.
4. Write user program according to users’ needs, and call the motor library functions in the program to quickly develop the motor control project.

Table 1. Correspondence between Flash size and ROM

Flash size	4032K	1024K	512K	448K	256K
IROM1(address)	0x8000000	0x8000000	0x8000000	0x8000000	0x8000000
IROM1(size)	0x3EF000	0xFF800	0x7F800	0x6F000	0x3F800
IROM2(address)	0x83EF000	0x80FF800	0x807F800	0x0806F000	0x803F800
IROM2(size)	0x1000	0x800	0x800	0x1000	0x800

Flash size	128K	64K	32K	16K
IROM1(address)	0x8000000	0x8000000	0x8000000	0x8000000
IROM1(size)	0x1FC00	0x0FC00	0x07C00	0x03C00
IROM2(address)	0x801FC00	0x800FC00	0x8007C00	0x8003C00
IROM2(size)	0x400	0x400	0x400	0x400

Note [1]: For Keil v5.33, the AT32 BSP source code does not support V6.15 compiler. Thus please use Keil compiler version 5 for compiling purposes.

Figure 1. AT32F413RCT7 ROM configuration (Keil)

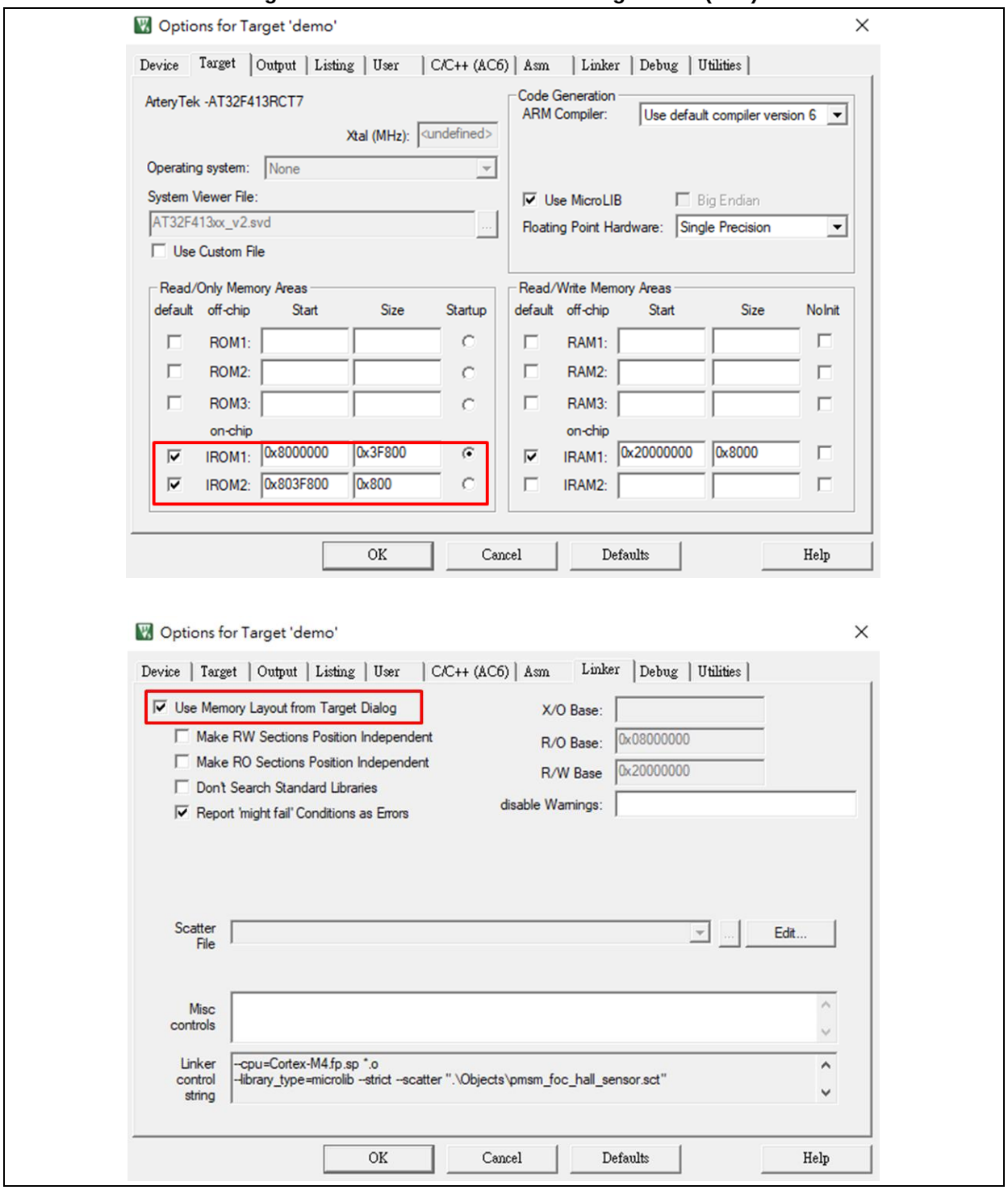
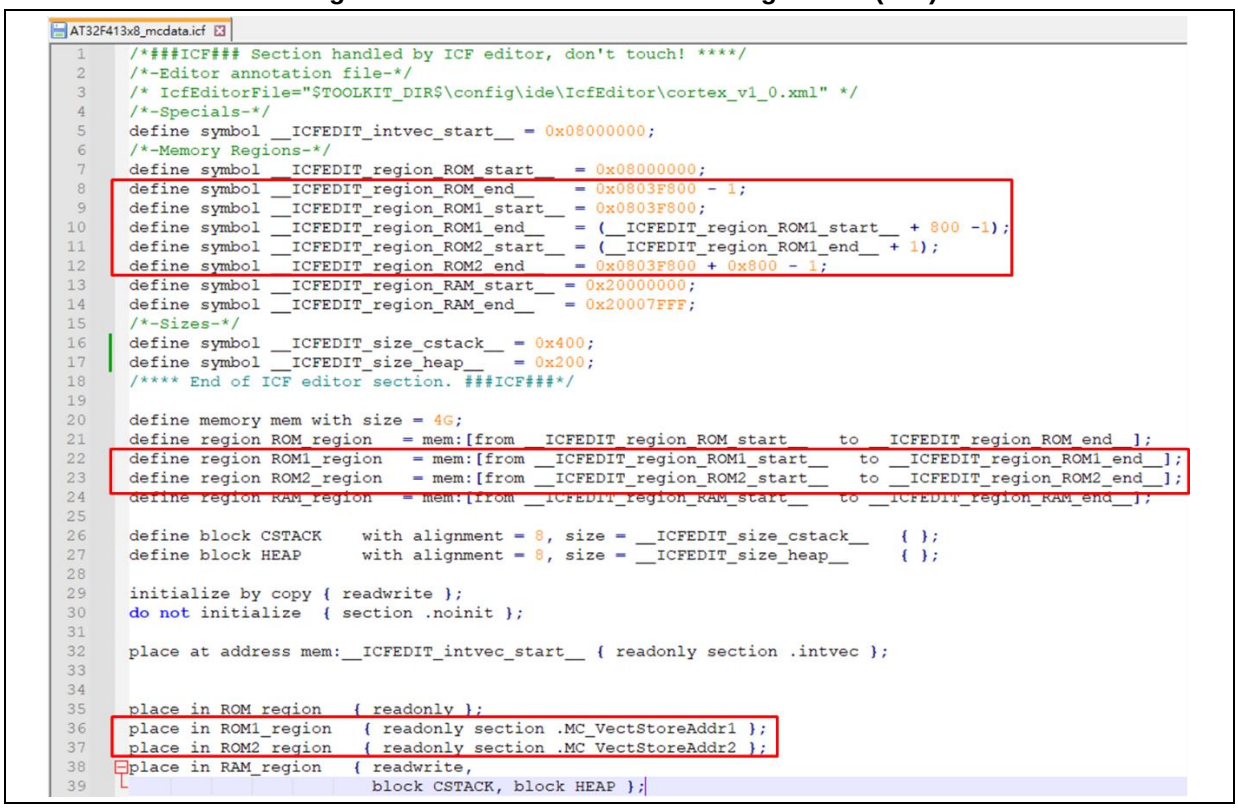


Figure 2. AT32F413RCT7 ROM configuration (AT32IDE)

AT32F413xC_FLASH.ld

```
_estack = 0x20008000;    /* end of RAM */

/* Generate a link error if heap and stack don't fit into RAM */
_Min_Heap_Size = 0x200;      /* required amount of heap  */
_Min_Stack_Size = 0x400; /* required amount of stack */

/* Specify the memory areas */
MEMORY
{
FLASH (rx)      : ORIGIN = 0x08000000, LENGTH = 0x3F800
MC_DATA (r)     : ORIGIN = 0x0803F800, LENGTH = 0x800
RAM (xrw)       : ORIGIN = 0x20000000, LENGTH = 32K
}

/* Define output sections */
SECTIONS
{
  /* The startup code goes first into FLASH */
  .isr_vector :
  {
    . = ALIGN(4);
    KEEP(*(.isr_vector)) /* Startup code */
    . = ALIGN(4);
  } >FLASH

  .mc_data :
  {
    . = ALIGN(4);
    . = ORIGIN(MC_DATA);
    _MC_VectStoreAddr1 = .;
    *(.MC_VectStoreAddr1);
    . = ALIGN(4);
    . = ORIGIN(MC_DATA) + 800;
    _MC_VectStoreAddr2 = .;
    *(.MC_VectStoreAddr2);
    . = ALIGN(4);
  } >MC_DATA

  /* The program code and other data goes into FLASH */
  .text :
  {
    . = ALIGN(4);
    *(.text)           /* .text sections (code) */
    *(.text*)          /* .text* sections (code) */
    *(.glue_7)         /* glue arm to thumb code */
    *(.glue_7t)        /* glue thumb to arm code */
    *(.eh_frame)
```

Figure 3. AT32F413RCT7 ROM configuration (IAR)



3 Motor control library files

Figure 4 illustrates the relationship between motor control functions, BSP, UI programs and hardware peripherals. In this motor control library architecture, the underlying hardware peripherals are controlled with BSP functions. The motor control functions and UI functions are built based on BSP and low-level functions. The user-defined programs are based on the motor control functions and UI functions. This organization makes it easier for users to call motor control functions to control MCU hardware peripherals and execute motor control programs. At the same time, it is also possible to obtain real-time motor control status or adjust motor control parameters by using UI control functions linked to the external PC UI software.

Figure 4. Motor control program architecture

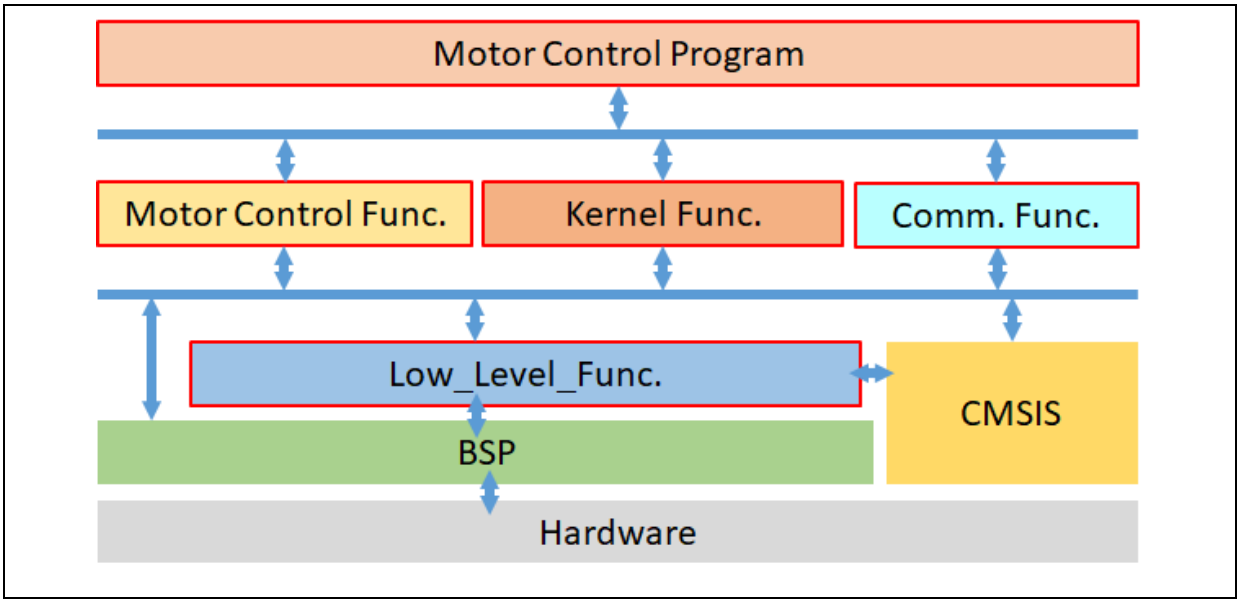


Figure 5 shows an overall structure of motor control library files. The "*motor_control_drive_param.h*" header file is used to configure drive mode, control parameters, motor parameters and board parameters, while the "*mc_hwio.h*" header file is used to set MCU peripheral parameters. All of these parameter settings are then integrated into the "*mc_foc_globals.h (mc_bldc_globals.h)*" and programmed with the initial variables through the "mc_foc_globals.c (mc_bldc_globals.c)". The "mc_hwio.c" can be used to initialize MCU peripherals.

Figure 5. Structure of motor control library files

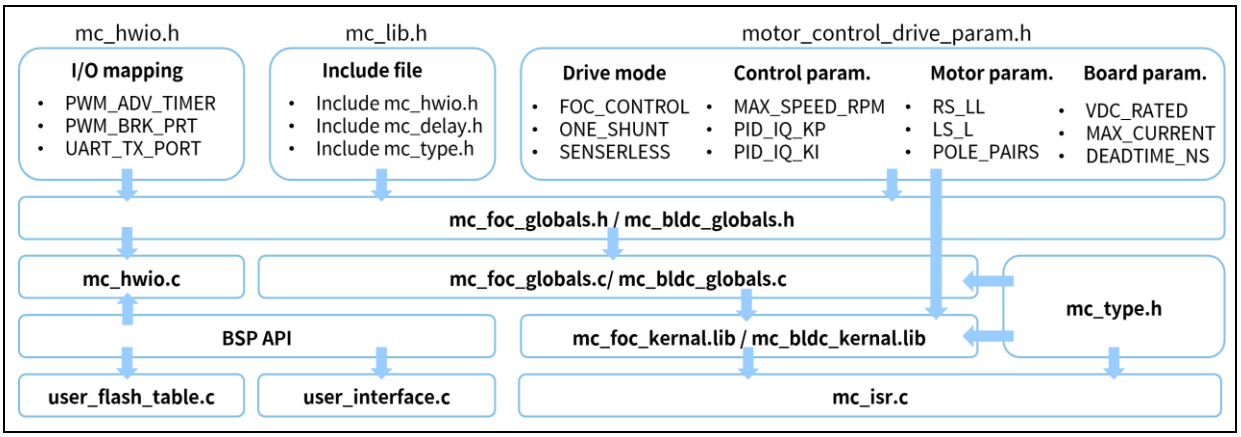


Table 2 gives a list of motor control library files.

Table 2. Motor control library file list

File name	Description
FOC/BLDC general files	
mc_lib.h	Header file management
motor_control_drive_param.h	Set motor drive mode (current sampling mode, sensored mode, ect.), control parameters, motor parameters and board parameters
mc_hwio.c	Hardware peripheral configuration
mc_hwio_v1.h	Set hardware IO macro definition (AT-MOTOR-EVB V1.x)
mc_hwio_v2.h	Set hardware IO macro definition (AT-MOTOR-EVB V2.x)
mc_isr.c	Motor control interrupt functions
mc_type.h	Global variables type and definition, enumeration definition
mc_delay.c	Delay functions
mc_delay.h	Declaration of delay functions
mc_comm_uart.c	Communication peripheral configuration
mc_comm_uart.h	Declaration and configuration of UART functions
mc_pid_control.c	PID controller functions
mc_pid_control.h	Declaration of PID controller functions
mc_curr_fdbk.c	Current sensing functions
mc_curr_fdbk.h	Declaration of current sensing functions
mc_math.c	Filter functions
mc_math.h	Declaration of filter functions
mc_hall.c	Hall sensor functions
mc_hall.h	Declaration of Hall sensor functions
FOC files	
mc_foc_kernal.lib	Motor control library core functions (for Keil environment only) (applicable to M4F MCU)
mc_foc_kernal_noFPU.lib	Motor control library core functions (for Keil environment only) (applicable to M4 MCU)
mc_foc_kernal_m0plus.lib	Motor control library core functions (for Keil environment only) (applicable to M0+ MCU)
libmc_foc_kernal.a	Motor control library core functions (for AT32IDE environment only) (applicable to M4F MCU)
libmc_foc_kernal_noFPU.a	Motor control library core functions (for AT32IDE environment only) (applicable to M4 MCU)
libmc_foc_kernal_m0plus.a	Motor control library core functions (for AT32IDE environment only) (applicable to M0+ MCU)
mc_foc_kernal.a	Motor control library core functions (for IAR system only) (applicable to M4F MCU)

mc_foc_kernal_noFPU.a	Motor control library core functions (for IAR system only) (applicable to M4 MCU)
mc_foc_kernal_m0plus.a	Motor control library core functions (for IAR system only) (applicable to M0+ MCU)
mc_foc_kernal.h	Declaration of motor control library core functions
motor_control_foc.c	Motor control functions
motor_control_foc.h	Declaration of motor control functions
mc_foc.c	Vector control functions
mc_foc.h	Declaration of vector control functions
mc_encoder.c	Encoder functions
mc_encoder.h	Declaration of encoder functions
mc_field_weakening.c	Field weakening functions
mc_field_weakening.h	Declaration of field weakening functions
mc_foc_sensorless.c	Sensorless functions
mc_foc_sensorless.h	Declaration of sensorless functions
mc_curr_decoupling.c	Current decouple control functions
mc_curr_decoupling.h	Declaration of current decouple control functions
MTPA_MTPV_table.c	Maximum Torque Per Ampere (MTPA)/Maximum Torque Per Volt (MTPV) field-weakening lookup table function
MTPA_MTPV_table.h	Declaration of Maximum Torque Per Ampere (MTPA)/Maximum Torque Per Volt (MTPV) field-weakening lookup table function
mc_foc_globals.c	Global variables definition and default value, global function declaration
mc_foc_globals.h	Global variables, global function declaration, macro definitions
user_interface_foc.c	Communication interface functions
user_interface_foc.h	Declaration of communication interface functions
mc_flash_data_table_foc.c	Write Flash data table
mc_flash_data_table_foc.h	Write Flash data table configuration
BLDC files	
mc_bldc_kernal.lib	Motor control library core functions (for Keil environment only) (applicable to M4F MCU)
mc_bldc_kernal_noFPU.lib	Motor control library core functions (for Keil environment only) (applicable to M4 MCU)
mc_bldc_kernal_m0plus.lib	Motor control library core functions (for Keil environment only) (applicable to M0+ MCU)
libmc_bldc_kernal.a	Motor control library core functions (for AT32IDE environment only) (applicable to M4F MCU)
libmc_bldc_kernal_noFPU.a	Motor control library core functions (for AT32IDE environment only) (applicable to M4 MCU)
libmc_bldc_kernal_m0plus.a	Motor control library core functions (for AT32IDE environment only) (applicable to M0+ MCU)
mc_bldc_kernal.a	Motor control library core functions (for IAR system only) (applicable to M4F MCU)

mc_bldc_kernal_noFPU.a	Motor control library core functions (for IAR system only) (applicable to M4 MCU)
mc_bldc_kernal_m0plus.a	Motor control library core functions (for IAR system only) (applicable to M0+ MCU)
mc_bldc_kernal.h	Declaration of motor control library core functions
motor_control_bldc.c	Motor control functions
motor_control_bldc.h	Declaration of motor control functions
mc_bldc.c	6-step square wave functions
mc_bldc.h	Declaration of 6-step square wave functions
mc_bldc_sensorless.c	Sensorless functions
mc_bldc_sensorless.h	Declaration of sensorless functions
mc_bldc_globals.c	Global variables definition and default value, global function definition
mc_bldc_globals.h	Global variables, global function declaration, macro definitions
user_interface_bldc.c	Communication interface functions
user_interface_bldc.h	Declaration of communication interface functions
mc_flash_data_table_bldc.c	Write Flash data table
mc_flash_data_table_bldc.h	Write Flash data table configuration

1) motor_control_drive_param.h

- This file is divided into four parts, which are used to configure drive mode, control parameters, motor parameters and board parameters.
- Table 3 presents the definition of drive modes. The user can configure the desired drive mode according to their hardware and motor configuration. For current sampling method, it is possible to select 3-shunt, 2-shunt or 1-shunt current sensing mode. For the positioning sensor, there are photoelectric incremental encoder, Hall sensor or sensorless available to choose from. In addition, the field weakening control is optional according to users' needs.

Table 3. Drive mode definitions

Definition	Description
FOC_CONTROL	FOC vector control mode
SIX_STEP_CONTROL	Six-step square wave control mode
THREE_SHUNT	Three-shunt current sampling
TWO_SHUNT	Two-shunt current sampling
ONE_SHUNT	Single-shunt current sampling
U_V_SHUNT	U & V shunt (for TWO_SHUNT only)
V_W_SHUNT	V & W shunt (for TWO_SHUNT only)
U_W_SHUNT	U & W shunt (for TWO_SHUNT only)
AT_MOTOR_EVB_V1	Applicable to AT-MOTOR-EVB V1.x
AT_MOTOR_EVB_V2	Applicable to AT-MOTOR-EVB V2.x
COMPLEMENT	Enable complementarity between low-side MOS transistor switch and upper-side PWM (see Figure 8)
GATE_DRIVER_LOW_SIDE_INVERT	Enable low-side MOS inverted output (see Figure 8)
EMF_COMPENSATE	EMF voltage compensate
INCREM_ENCODER	Photoelectric incremental encoder
MAGNET_ENCODER_W_ABZ	Magnetic Encoder with ABZ Signals
MAGNET_ENCODER_WO_ABZ	Magnetic Encoder without ABZ Signals
REVERSE_ENCODER_COUNT	Reverse encoder counting (photoelectric incremental encoder)
ABZ	AB signal with zero position (photoelectric incremental encoder)
AB	AB signal without zero position (photoelectric incremental encoder)
M_METHOD	M method for speed measurement (photoelectric incremental encoder)
MT_METHOD	M/T method for speed measurement (photoelectric incremental encoder)
ANGLE_CALIBRATION	Magnetic encoder angle self-calibration
HALL_SENSORS	Hall sensor (general-purpose)

LOW_SPEED_VOLT_CTRL	Low-speed voltage control
WITHOUT_CURRENT_CTRL	Voltage control without current loop
PHASE_ADVANCE	Phase advance (six-step square wave sensorless control)
LARGE_COGGING	Large cogging torque (six-step square wave sensorless control)
FIELD_WEAKENING	Field weakening control
SENSORLESS	Sensorless control mode (general-purpose)
OPENLOOP_STARTUP	Open loop startup in sensorless control mode (general-purpose)
ALIGN_AND_GO_STARTUP	Align and go startup in sensorless control mode (general-purpose)
INIT_ANGLE_STARTUP	Initial angle detection startup in sensorless control mode (general purpose)
VOLT_SENSE	Voltage sensing in sensorless vector control mode
WIND_SENSE	Sensorless Upwind/Downwind Startup
CONST_CURRENT_START	Constant current startup (sensorless control)
CONST_VOLTAGE_START	Constant voltage startup (sensorless control)
BLDC_SENSORLESS_ADC	BEMF detection with ADC in six-step square wave sensorless control mode
BLDC_SENSORLESS_COMP	BEMF detection with comparator in six-step square wave sensorless control mode
EMF_CONTINOUS_SAMPLE	Continuous sampling mode for BEMF zero-crossing detection (six-step square wave sensorless comparator mode)
SPECIFIC_EACH_EMF_PIN	Shield signals from other comparators (six-step square wave sensorless mode with comparator)
EMF_PULL_UP	ARTERY's proprietary pull-up circuit dedicated to enhance BEMF sampling and analysis with ADC in low-speed mode (Patent) (BEMF detection with ADC in six-step square wave sensorless mode)
CURRENT_LP_FILTER	Get d/q axis current low-pass filter signals (vector control)
AC_CURRENT_LP_FILTER	Get A/B/C Phase Low-Pass Filtered Current Signals (Vector Control/FOC)
INTERNAL_CLOCK_SOURCE	Use MCU internal oscillator as a clock source
E_BIKE_SCOOTER	E-bike/scooter mode
DC_CURRENT_LIMIT	DC current limit (for E_BIKE_SCOOTER only)
RDS_AUTO_CALIBRATION	MOS's $R_{DS(on)}$ auto calibration (for E_BIKE_SCOOTER only)
MOTOR_PARAM_IDENTIFY	Motor winding parameters identification
CURRENT_DECOUPLE_CTRL	Current decouple control (AT32L021 series Excluded)
TORQUE_CTRL_WITH_SPEED_LIMIT	Torque (current) control, including the maximum speed limitation

BRAKING_RESISTOR	Use an external braking resistor
SPMSM	Surface-mounted permanent magnet synchronous motor
IPMSM	Interior Permanent Magnet Synchronous Motor
IPM_MTPA_MTPV_TABLE	MTPA/MTPV Table Generation (IPMSM-Specific) (AT32L021 series Excluded)
IPM_MTPA_MTPV_CTRL	MTPA/MTPV lookup table control (IPMSM-Specific) (AT32L021 series Excluded)
USE_MOTOR_MONITOR	Artery motor UI tool
MOS_RDS_SHUNT	Current Sampling Utilizing Low-Side MOSFET's $R_{DS(on)}$
TWO_ADC_CONVERTERS	Current Sampling with Dual-ADC Synchronization (Applicable for MCUs with dual or more ADCs; Not compatible with ONE_SHUNT topology)

- Table 4 presents motor control parameters. The user can define or tune corresponding parameters according to actual requirements, such as, current d-axis, q-axis PI controller, speed PI controller.

Table 4. Control parameter definitions

Definition	Description
BLDC/FOC general definitions	
PWM_FREQ	PWM output frequency (unit: Hz)
MOTOR_CONTROL_MODE	Motor control mode (speed control, torque control, etc.; see *motor_control_mode* in *type.h* for details)
CTRL_SOURCE	Configure command control source (external source control/software control)
UI_UART_BAUDRATE	Baud rate of UI communication serial interface
TUNE_TARGET_CURRENT	Tune target current value of PI parameters (unit: ampere)
TUNE_CURRENT_TOTAL_PERIOD	Tune total current pulse period of PI parameters (unit: ms)
TUNE_CURRENT_STEP_PERIOD	Tune current pulse step period of PI parameters (unit: ms)
MIN_SPEED_RPM	Minimum motor speed (unit: rpm)
MAX_SPEED_RPM	Maximum motor speed (unit: rpm)
MIN_SENSE_SPEED	Minimum Detectable Speed (Unit: rpm) (For Hall Sensors Only)
ACC_SPD_SLOPE	Slope of acceleration (unit: rpm/ms, when systick frequency =1kHz)
DEC_SPD_SLOPE	Slope of deceleration (unit: rpm/ms, when systick frequency =1kHz)
SP_MAX_VOLT	Maximum voltage of external command source (unit: voltage)
SP_THRESHOLD	Threshold voltage of external command source (unit: voltage)
SP_RUN_VALUE	Minimum voltage to start running in external source mode (unit: voltage)

Definition	Description
SP_STOP_VALUE	Maximum voltage to stop running in external source mode (unit: voltage)
PID_SPD_KP_DIV_DIV	Speed control proportional gain divisor (Q16 mode)
PID_SPD_KI_DIV_DIV	Speed control integral gain divisor (Q16 mode)
PID_SPD_KP_DEFUALT	Speed control proportional gain (Q15 mode)
PID_SPD_KI_DEFUALT	Speed control integral gain (Q15 mode)
HYSTERESIS_LOW_SPEED	Minimum speed from current control to switch back to voltage control in low-speed voltage control mode (unit: rpm) (for LOW_SPEED_VOLT_CTRL only)
HYSTERESIS_HIGH_SPEED	Maximum speed from voltage control to switch back to current control in low-speed voltage control mode (unit: rpm) (for LOW_SPEED_VOLT_CTRL only)
PID_SPD_VOLT_KP_DEFUALT	Low-speed voltage control proportional gain (Q15 mode) (for LOW_SPEED_VOLT_CTRL and WITHOUT_CURRENT_CTRL only)
PID_SPD_VOLT_KI_DEFUALT	Low-speed voltage control integral gain (Q15 mode) (for LOW_SPEED_VOLT_CTRL and WITHOUT_CURRENT_CTRL only)
PID_SPD_VOLT_KP_GAIN_DIV	Low-speed voltage control proportional gain divisor (Q16 mode) (for LOW_SPEED_VOLT_CTRL and WITHOUT_CURRENT_CTRL only)
PID_SPD_VOLT_KI_GAIN_DIV	Low-speed voltage control integral gain divisor (Q16 mode) (for LOW_SPEED_VOLT_CTRL and WITHOUT_CURRENT_CTRL only)
AUTO_TUNE_CURR_BANDWIDTH	Current PI controller bandwidth (used for self-tuning of current PI controller parameters)
FOC definitions	
SPEED_LOOP_FREQ	Speed Loop Control Frequency (unit: Hz)
STABLE_SPEED_RPM	Stable motor speed (unit: rpm) (for hall sensor use only)
SLICK_SPEED_RPM	Motor slick speed (unit: rpm) (for hall sensor use only)
MIN_POSCTL_SPD	Minimum speed for position loop control (unit: rpm)
POSITION_LOOP_FREQ	Position loop control frequency (unit: Hz)
MAX_POSITION_ANGLE	Maximum position angle (unit: Degree)
MIN_POSITION_ANGLE	Minimum position angle (unit: Degree)
CMD_TO_VAL_GAP	Define the counter gap value between rotor position and its target to use PID_POS_KI_DEFUALT_STABLE
SMALL_POS_CMD_GAP	When the rotor position difference is within a certain range of the target position, the minimum speed command (MIN_POSCTL_SPD) is used for position loop control.
ROTOR_LOCK_ANGLE_GAP	When the rotor position difference is within a certain electrical

Definition	Description
	angle of the target position, the rotor position is locked (for hall sensor only)
PID_POS_KP_DEFUALT	Rotor position control proportional gain (Q15 mode)
PID_POS_KI_DEFUALT	Rotor position control integral gain (Q15 mode)
PID_POS_KI_DEFUALT_STABLE	Rotor position control integral gain (Q15 mode) (rotor position close to target position)
PID_POS_KD_DEFUALT	Rotor position control derivative gain (Q15 mode)
PID_POS_KP_GAIN_DIV	Rotor position control proportional gain divisor (Q16 mode)
PID_POS_KI_GAIN_DIV	Rotor position control integral gain divisor (Q16 mode)
PID_POS_KD_GAIN_DIV	Rotor position control derivative gain divisor (Q16 mode)
PID_ID_KP_DEFUALT	d-axis current control proportional gain (Q15 mode)
PID_ID_KI_DEFUALT	d-axis current control integral gain (Q15 mode)
PID_ID_KP_GAIN_DIV	d-axis current control proportional gain divisor (Q16 mode)
PID_ID_KI_GAIN_DIV	d-axis current control integral gain divisor (Q16 mode)
PID_IQ_KP_DEFUALT	q-axis current control proportional gain (Q15 mode)
PID_IQ_KI_DEFUALT	q-axis current control integral gain (Q15 mode)
PID_IQ_KP_GAIN_DIV	q-axis current control proportional gain divisor (Q16 mode)
PID_IQ_KI_GAIN_DIV	q-axis current control integral gain divisor (Q16 mode)
OLC_ANGLE_INC	Open loop angle incremental at each PWM output frequency
OLC_VOLT	Open loop voltage output (unit: voltage)
ALIGN_VOLT	Encoder align voltage (unit: voltage)
ENC_OFFSET	Encoder align offset (unit: resolution or count per revolution)
ENC_OLC_VOLT	Magnetic Encoder Angle Calibration Open-Loop Voltage (unit: volt)
ENC_OLC_ANGLE_INC	Magnetic Encoder Angle Calibration Open-Loop Angle Increment (per PWM Cycle)
ENC_OLC_MOTOR_REV	Magnetic Encoder Angle Calibration Forward/Reverse Rotation Cycles (unit: revolution)
LEARN_OLC_VOLT	HALL self-learning open loop voltage (unit: voltage)
LEARN_OLC_ANGLE_INC	HALL self-learning angle incremental (per PWM output frequency)
LEARN_TIME	HALL self-learning time (unit: ms)
LEARN_ALIGN_TIME	HALL self-learning align time (unit: ms)
FW_MAX_ID_CURR	Maximum d-axis current in field weakening control (unit: ampere)
FW_KP_GAIN	Field weakening control proportional gain (Q15 mode)
FW_KI_GAIN	Field weakening control integral gain (Q15 mode)
FW_KP_GAIN_DIV	Field weakening control proportional gain divisor (Q16 mode)

Definition	Description
FW_KI_GAIN_DIV	Field weakening control integral gain divisor (Q16 mode)
CURR_LP_BANDWIDTH	d/q-axis current low-pass filter band width (unit: Hz)
AC_CURR_LP_BANDWIDTH	a/b/c phase current low-pass filter band width (unit: Hz)
OBS_SPD_LP_BANDWIDTH	Speed low-pass filter band width of observer (unit: Hz)
OBS_GAIN1	Observer gain factor 1 (Q15 mode)
OBS_GAIN2	Observer gain factor 2 (Q15 mode)
PLL_KP_GAIN	PLL control proportional gain (Q15 mode)
PLL_KI_GAIN	PLL control integral gain (Q15 mode)
PLL_KP_GAIN_DIV	PLL control proportional gain divisor (Q16 mode)
PLL_KI_GAIN_DIV	PLL control integral gain divisor (Q16 mode)
STARTUP_MAX_SPD	Open loop maximum speed before entering closed loop (unit: RPM)
STARTUP_CURRENT	Current command before entering closed loop (unit: ampere)
STARTUP_VOLTAGE	Voltage command before entering closed loop (unit: voltage)
STARTUP_OL_SLOPE	Open loop startup acceleration (unit: rpm/s)
STARTUP_ALIGN_TIME	Rotor alignment time before startup (unit: ms)
STARTUP_START_TIME	Startup time after rotor align and go (entering closed loop) (unit: ms)
DETECT_PULSE_WIDTH	Pulse width for rotor initial angle detection (unit: us)
DETECT_DELAY_TIME	Detect motor reverse time in sensorless startup delay (unit: ms)
DETECT_REVERSE_SPEED	Detect motor reverse speed in sensorless startup mode (unit: rpm)
DETECT_MAX_SPEED	Detect the motor maximum forward speed in sensorless startup mode (unit: rpm)
WIND_DETECT_TIME	Upwind/Downwind detection time (unit: ms)
TAILWIND_SPEED	Minimum speed for downwind closed-loop engagement (unit: rpm)
HEADWIND_BRAKE_TIME	Upwind braking time (unit: ms)
REVERSE_MAX_SPEED_RPM	Maximum reverse speed (unit: rpm) (for E_BIKE_SCOOTER only)
REVERSE_CURRENT	Reverse current command (unit: ampere) (for E_BIKE_SCOOTER only)
BRAKING_CURRENT	Brake current command (unit: ampere) (for E_BIKE_SCOOTER only)
MAX_LOCK_CURRENT	Maximum Parking/Theft-Deterrent Current Command (unit: ampere) (Dedicated for E_BIKE_SCOOTER)
ANTI_THEFT_INIT_CURRENT	Anti-theft Initial Current Value (unit: ampere) (E-BIKE_SCOOTER only)

Definition	Description
ANTI_THEFT_INC_CURRENT	Anti-theft Current Increment per Millisecond (unit: ampere) (E-BIKE_SCOOTER only)
ANTI_THEFT_DEC_CURRENT	Anti-theft Current Decrement per Millisecond (unit: ampere) (E-BIKE_SCOOTER only)
PARKING_LOCK_INIT_CURRENT	Parking Initial Current Value (unit: ampere) (E-BIKE_SCOOTER only)
PARKING_LOCK_INC_CURRENT	Parking Current Increment per Millisecond (unit: ampere) (E-BIKE_SCOOTER only)
PARKING_LOCK_DEC_CURRENT	Parking Current Decrement per Millisecond (unit: ampere) (E-BIKE_SCOOTER only)
ANTI_THEFT_LOCK_TIME	Anti-theft Maximum Current Lock Release Time (unit: ms) (E-BIKE_SCOOTER only)
PARKING_LOCK_TIME	Parking Maximum Current Lock Release Time (unit: ms) (E-BIKE_SCOOTER only)
VD_VOLT_LOW_SPD	D-axis voltage command during low-speed voltage control (unit: voltage) (Exclusive to LOW_SPEED_VOLT_CTRL Mode)
HIGH_PWM_DUTY_CYCLE	Dynamic Adjustment of Current Sampling Position at High PWM Duty Cycle (TWO_ADC_CONVERTERS only)
MAX_DUTY_PCT	Maximum PWM Duty Cycle Setting
BLDC definitions	
I_SAMP_MIN_TIME	Minimum sampling time to obtain ADC value of current sensing signal (unit: nsec)
I_SAMP_DELAY	Driver signal delay time plus current sensing delay time (unit: nsec)
SENSE_HALL_TIMES	Set phase-change times before open loop enters closed loop (unit: times) (for BLDC sensorless only)
REBOOT_PERIOD_MS	Set a restart time when a startup failed (unit: ms) (for BLDC sensorless only)
START_CURRENT	Current value for constant current startup (unit: ampere) (for BLDC sensorless only)
START_VOLTAGE	Voltage value for constant voltage startup (unit: V)
SPEED_FILTER_TIMES	Speed moving average times (unit: times)
SPD_LP_BANDWIDTH	Speed low-pass filter band width (unit: Hz)
PWM_IN_FILTER_TIMES	PWM pulse width low-pass filter times (unit: times)
SPD_CMD_FILTER_TIMES	Speed command low-pass filter times (unit: times)
MAX_CCW_SPEED_RPM	Maximum motor reverse speed (unit: rpm)
PWM_DUTY_TMR_DIV	Clock divider for detecting PWM pulse width input
PWM_IN_MAX_WIDTH	Maximum width of PWM input (unit: usec)
PWM_IN_NEUTRAL_WIDTH	Neutral width of PWM input (unit: usec)

Definition	Description
PWM_IN_MIN_WIDTH	Minimum width of PWM input (unit: usec)
DEADZONE_PULSEWIDTH	Dead-zone width of PWM input (unit: usec)
TOLERANCE_PULSEWIDTH	Tolerance width of PWM input (unit: usec)
SPD_CURVE_SWITCH_WIDTH	Switching interval width of PWM input (unit: usec)
SWITCH_SPD_RPM	Speed at the switching point of PWM input (unit: RPM)
BRAKE_DUTY	Brake PWM duty cycle (unit: PWM duty%)
DRAG_BRAKE_DUTY	Drag brake PWM duty cycle (unit: PWM duty%)
TONE_FREQ	Warning tone frequency (unit: Hz)
TONE_AMP	Warning tone volume
TONE_PERIOD	Warning tone duration
OLC_INIT_SPD	Open loop initial speed (unit: rpm)
OLC_FINAL_SPD	Open loop final speed (unit: rpm)
OLC_TIMES	Set the times from initial speed to rise up to final speed (unit: times)
OLC_INIT_VOLT	Open loop initial voltage (unit: V)
OLC_VOLT_INC	Open loop incremental voltage (unit: V)
OLC_STARTUP_PERIOD	Set the period from open loop startup to closed loop (unit: ms) (for BLDC sensorless use only)
LOCK_VOLT	Rotor alignment voltage before startup (unit: V) (for BLDC sensorless use only)
LOCK_PERIOD	Rotor alignment time before startup (unit: ms) (for BLDC sensorless use only)
LEARN_TIME	HALL self-learning time (unit: msec)
LEARN_ALIGN_TIME	HALL self-learning align time (unit: msec)
PID_IS_KP_DIV	Bus current control proportional gain divisor (Q16 mode)
PID_IS_KI_DIV	Bus current control integral gain divisor (Q16 mode)
PID_IS_KP_DEFUALT	Bus current control proportional gain (Q15 mode)
PID_IS_KI_DEFUALT	Bus current control integral gain (Q15 mode)
EMF_CHANGE_PERCENT_H	Duty value for BEMF zero-crossing detection to switch from low speed mode to high speed mode (unit: PWM duty %) (for sensorless BLDC only)
EMF_CHANGE_PERCENT_L	Duty value for BEMF zero-crossing detection to switch from high speed mode to low speed mode (unit: PWM duty %)) (for sensorless BLDC only)
MIN_SAMPLE_INTERVAL_US	Minimum sampling interval for BEMF (> 1us is recommended) (unit: usec)
EMF_SAMPLE_LEAD_PWM	BEMF zero-crossing sampling point DUTY value in low speed range (unit: PWM timer timing base) (for sensorless BLDC

Definition	Description
	only)
EMF_HIGH_SPD_SAMPLE_CNT	BEMF zero-crossing sampling point DUTY value in high speed range (unit: PWM timer timing base) (for sensorless BLDC only)
EMF_PHASE_ADV_SPD	Phase advance maximum speed (unit: rpm)
EMF_MIN_DELAY_US	Phase advance minimum delay (unit: usec)
EMF_AVOID_NOISE_INIT_PERIOD	First delay time for detecting BEMF zero-crossing point (unit: ms)
EMF_SAMPLE_INTERVAL	Sampling interval in continuous sampling mode (unit: PWM timer timing base) (for sensorless BLDC only)
EMF_LOW_SPD_CONT_SAMPLE_END	Low-speed sampling end point in continuous sampling mode (unit: PWM timer timing base) (for sensorless BLDC only)
EMF_AVOID_NOISE_TIMES	Delay time for BEMF zero-crossing to avoid phase change noise (unit: PWM timer timing base)

- Table 5 presents the definitions relating to driver functions, such as dead time, current sensing resistor, and current amplifier gain, etc.

Table 5. Driver parameter definitions

Definition	Description
VDC_RATED	DC-BUS voltage (unit: voltage)
BAT_LOW_VOLT	Lowest battery voltage (unit: voltage)
V_SENSE_GAIN	Voltage feedback ratio
MAX_CURRENT	Maximum output current of motor controller (unit: ampere)
MIN_CURRENT	Minimum output current of motor controller (unit: ampere)
CURRENT_SPAN_SHIFT	Number of shifts for current per-unit normalization
ADC_REFERENCE_VOLT	ADC reference voltage (unit: voltage)
ADC_DIGITAL_SCALE_12BITS	ADC resolution
SYSTEM_CORE_CLOCK	System frequency (unit: Hz)
TMR_CLK	Timer clock frequency (unit: Hz)
CHANGE_PHASE_TMR_DIV	Phase change clock frequency division (for six-step square wave sensorless mode)
DEADTIME_CLK_SFT_BITS	Dead time frequency division shift bits
DEADTIME_NS	Dead time (unit: ns)
MIN_INTERVAL_TIME	Minimum interval between two-phase signals when PWM shift (unit: ns)
ADC_TRIG_DELAY_TIME	Current sampling delay time (unit: us)
ADC_TRIG_LAG_TIME	ADC trigger lag time for current sensing (Two-Phase Low-Level Condition) (Unit: ns)(only for

	TWO_ADC_CONVERTERS && THREE_SHUNT)
SW_OP_INP_MODE_LEAD_TIME	OP Input Mode Switching Lead Time Prior to ADC Trigger (for chips with internal OP like M412)
DC_MAX_CURRENT	Bus maximum current (unit: ampere) (for DC_CURRENT_LIMIT only)
R_SHUNT	Shunt resistor (unit: Ω)
OP_GAIN	Current amplifier gain
CURR_OFFSET_VOLT	Zero current offset (unit: voltage)
RDC_SHUNT	DC shunt resistance (unit: Ω) (for DC_CURRENT_LIMIT only)
DC_OP_GAIN	DC current amplifier gain (for DC_CURRENT_LIMIT only)
IDC_OFFSET_VOLT	Zero DC current offset (unit: voltage) (for DC_CURRENT_LIMIT only)
EMF_SENSE_GAIN	BEMF feedback ratio
GATE_DELAY_TIME	Gate delay time (unit: usec)
EMF_SIG_RISING_TIME	BEMF signal rising time (unit: usec)
EMF_SIG_FALLING_TIME	BEMF signal falling time (unit: usec)
EMF_RISE_BLANK_TIME	PWM On-state Blanking Window Size (unit: usec) (Dedicated for BLDC Sensorless Back-EMF Detection via Internal Comparator)
EMF_FALL_BLANK_TIME_HIGH_SPD	Blanking window duration when PWM is OFF during high-speed operation (Unit: μs) (For sensorFor sensorless BLDC internal comparator back-EMF detection method)
EMF_FALL_BLANK_TIME_LOW_SPD	Blanking window duration when PWM is OFF during low-speed operation (Unit: μs) (For sensorless BLDC internal comparator back-EMF detection method)
EMF_BLANK_TIME_CHANGED_RPM	RPM threshold for switching between high-speed and low-speed blanking windows during PWM OFF state (Unit: rpm) (For sensorless BLDC internal comparator back-EMF detection method)
EMF_MIN_VALUE	Minimum Back-EMF Threshold for Motor Stop Detection (ADC Reading) (Dedicated for BLDC Sensorless Back-EMF Detection via Internal Comparator)
OVER_CURRENT_SW	Software Overcurrent Threshold (unit: ampere)
DAC_VREF_SOURCE	DAC Source Selection for Internal OP Overcurrent Protection (Dedicated to chips with internal OP, e.g., M412 series)
OCP_CURRENT	Hardware Overcurrent Threshold (unit: ampere)
BUS_CURR_CMP_OCP_VOLT	Internal OP Overcurrent Protection Voltage (unit: volt) (Dedicated to chips with internal operational amplifier, e.g., M412 series)

TMR_BRK_FILTER_COUNT	Overcurrent Protection Break Input Filter Count (Dedicated to chips with break input function, e.g., M412 series)
OVER_VOLT_THRESHOLD	Over-voltage threshold point (unit: voltage)
UNDER_VOLT_THRESHOLD	Under-voltage threshold point (unit: voltage)
V0_V	Parameter V0 of the approximate curve of NTC temperature and voltage relationship (note [2])
T0_C	Parameter T0 of the approximate curve of NTC temperature and voltage relationship (note [2])
dV_dT	Parameter dV/dT of the approximate curve of NTC temperature and voltage relationship (note [2])
OVER_TEMP_THRESHOLD	Over-temperature threshold point (unit: Celsius degrees)
MC_ERROR_MASK	Error detection mask

Note [2]: Approximate curve equation for voltage-temperature relation is V[V]=V0+dV/dT[V/Celsius](T-T0)[Celsius].*

- Table 6 presents the definitions relating to motor parameters, such as pole-pairs, encoder parameter, Hall sensor parameter, etc.

Table 6. Motor parameter macro definitions

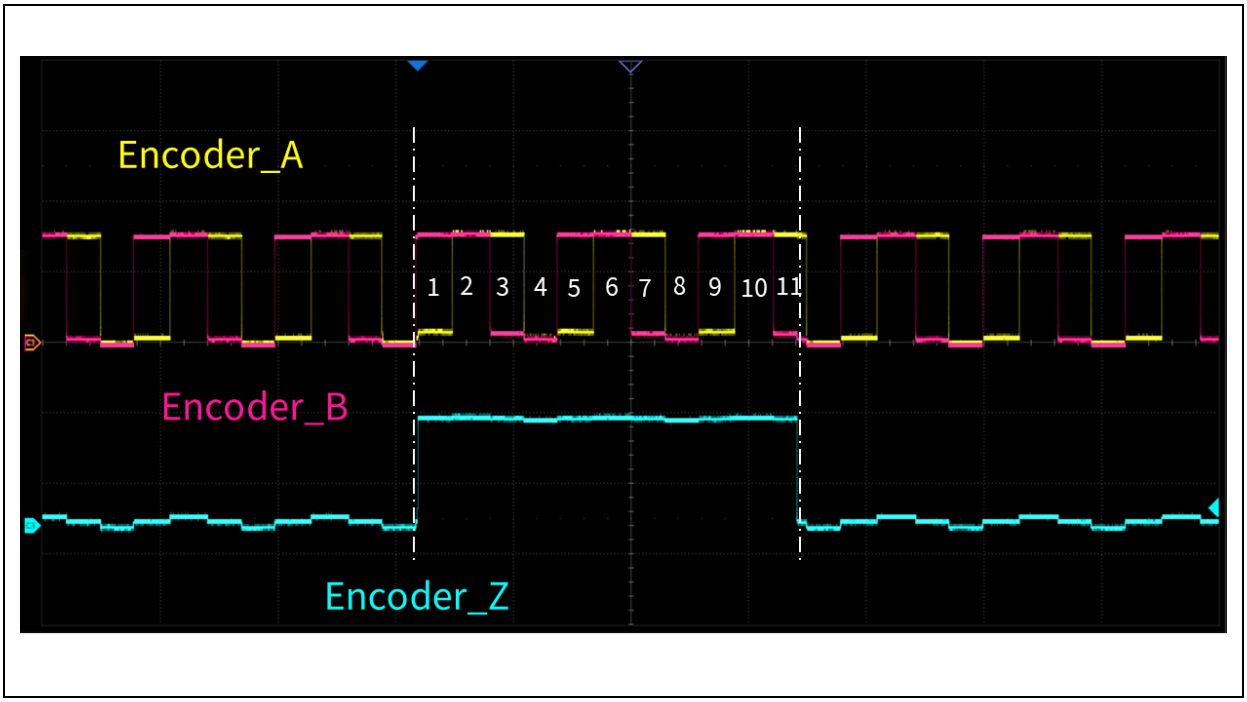
Definition	Description
BLDC/FOC general parameters	
RS_LL	Motor line-to-line winding resistance (unit: Ω)
LS_LL	Motor line-to-line winding inductance (unit: H)
POLE_PAIRS	Number of pole-pairs
KE	Back EMF constant (KE) (unit: V/rpm)
NOMINAL_CURRENT	Nominal current (unit: ampere)
HALL_LEARN_DIR	Motor rotation direction after hall self-learning procedure
HALL_LEARN_0_STATE	The state 0 after Hall self-learning (FOC: Hall state at electrical angle 0 degree) note[4]; BLDC: output V-phase upper side PWM and W-phase lower side ON)
HALL_LEARN_1_STATE	The state 1 after Hall self-learning (FOC: Hall state at electrical angle 60 degree) note[4]; BLDC: V-phase upper side PWM output and U-phase lower side ON)
HALL_LEARN_2_STATE	The state 2 after Hall self-learning (FOC: Hall state at electrical angle 120 degree) note[4]; BLDC: W-phase upper side PWM output and U-phase lower side ON)
HALL_LEARN_3_STATE	The state 3 after Hall self-learning (FOC: Hall state at electrical angle 180 degree) note[4]; BLDC: W-phase upper side PWM output and V-phase lower side ON)
HALL_LEARN_4_STATE	The state 4 after Hall self-learning (FOC: Hall state at electrical angle 240 degree) note[4]; BLDC: U-phase upper side PWM output and V-phase lower side ON)
HALL_LEARN_5_STATE	The state 5 after Hall self-learning (FOC: Hall state at electrical angle 300 degree) note[4]; BLDC: U-phase upper side PWM output and W-phase lower side ON)
FOC parameters	
LD_LQ_RATIO	Ld to Lq ratio
ABS_ENCODER_RES	Magnetic Encoder Resolution (unit: bits; two to the power of bit)
ABS_ENCODER_IIF_RES	Magnetic Encoder Incremental Interface Resolution (unit: count per revolution)
ABS_ENCODER_SAMPLE_TIME	Magnetic Encoder SPI Interface Data Transmit Time (unit: μs)
ENCODER_PPR	Pulses per revolution of encoder (unit: pulse per revolution)
ENC_IDX_COUNT	Encoder index signal width (unit: count) (note [3])
ENC_STALL_TIME	Encoder stall time (unit: ms)
BLDC sensorless parameters	
ANGLE_INIT_DETECT_DUTY	PWM DUTY value at initial angle detection (unit: PWM timer

Definition	Description
	timing base)
ANGLE_INIT_I_DIFF	Initial angle difference

Note [3]: This is applicable to ABZ mode of photoelectric incremental encoder.

Figure 6 presents the diagram of JK42BLS01-X056ED encoder with ABZ signals. If the width gap between Z signal's rising edge and its falling edge equals to 11 counts of AB signals, the ENC_IDX_COUNT is set to 11 (usually photoelectric encoder has one or two counts).

Figure 6. Encoder ABZ signal relationship

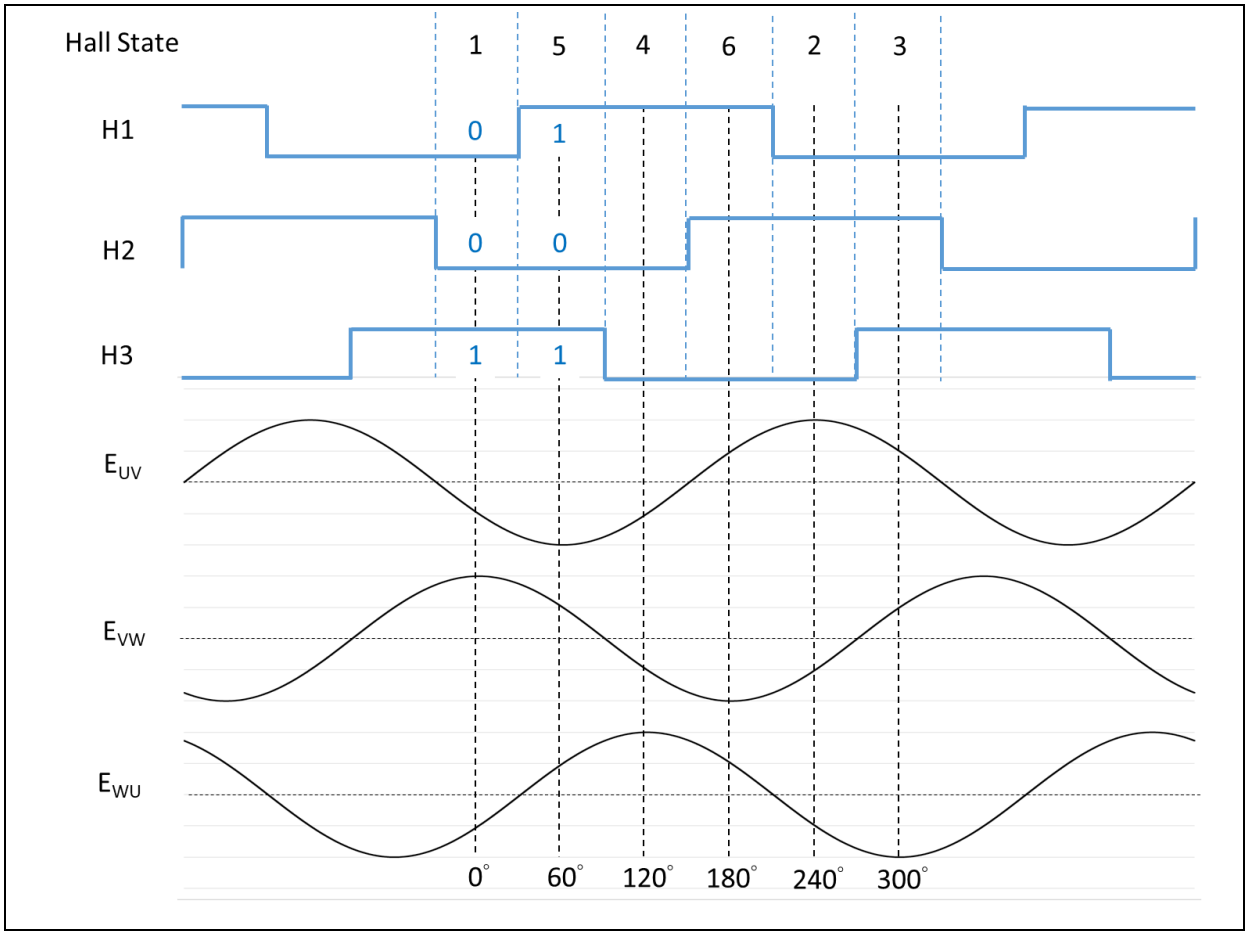


Note [4]: Hall state sequence and corresponding electrical angle are configurable according to BEMF and hall state.

Figure 7 presents the relationship between BLDC (JK42BLS01-X056ED) BEMF, Hall state and electrical angle (HALL_LEARN_DIR=0). The line-to-line BEMF for three-phase motor should be in consistent with this diagram. The zero degree of angle corresponds to the maximum BEMF between VW lines, while 60-degree angle corresponds to the minimum BEMF between UV lines, and so on. Different hall state values indicate different electrical angles. However, there is no need for AT32 MCU users to spend time finding this relationship as AT32 motor control library comes with a Hall phase sequence self-learning feature capable of providing corresponding Hall state values. This is achieved by writing Hall state values after Hall phase sequence self-learning routine into the corresponding hall sensor macro definitions.

Refer to *AN0167_AT32_PMSM_FOC_Hall_Demo* or *AN0194_AT32_PMSM_FOC_Hall_Demo (E-Bike-Scooter)* for details.

Figure 7. Relationship between PMSM BEMF, Hall state and electrical angle (HALL_LEARN_DIR=0)

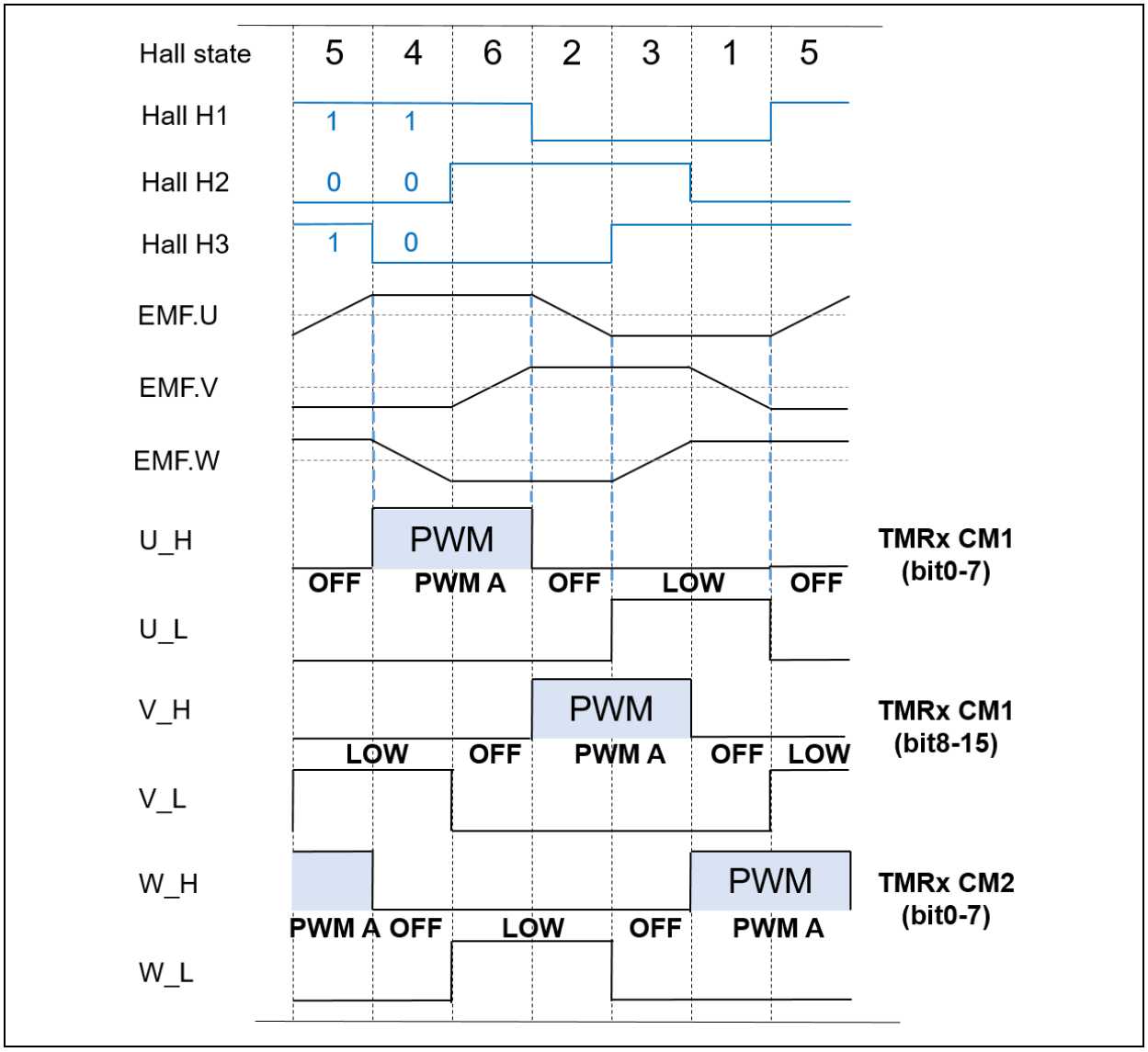


Note [5]: It is possible to set different MOS high-side and low-side ON/OFF state according to BEMF and hall state.

Figure 8 illustrates the relationship between BLDC BEMF, Hall state and MOS ON when HALL_LEARN_DIR=0. For example, we can see from Figure 8 that hall state 2 corresponds to the maximum BEMF for V-phase and the minimum BEMF for W-phase, and so the HALL_LEARN_0_STATE is set to 2, and so on and so forth. However, there is no need for AT32 MCU users to spend time finding this relationship as AT32 motor control library comes with a Hall phase sequence self-learning feature capable of providing corresponding Hall state values. This is achieved by writing Hall state values after Hall phase sequence self-learning routine into the corresponding hall sensor macro definitions. For details, please refer to *AN0169_AT32_BLDC_Hall_1-shunt_Demo*.

The following PWM output state refers to the scenario when using a gate driver with non-inverted lower-side input signals, and disabling lower-side PWM complementarity upon upper-side PWM toggle. If the user selects a gate driver with inverted lower-side input and enables PWM complementarity output upon upper-side PWM toggle, it is possible to change output mode with the motor_control_drive_param.h function, for example, our three six-step square wave control demo projects use the gate driver with inverted lower-side output and PWM complementary output enabled.

Figure 8. Relationship between BLDC BEMF, Hall state and MOS ON/OFF (HALL_LEARN_DIR=0)



2) mc_hwio.h file

- This file is used to configure macro definitions according to the user hardware IOs and peripherals. It also includes the declaration of the mc_hwio.c file functions.

3) mc_hwio.c file

- This file is used to configure peripherals such as TMR, ADC, DMA and GPIO, according to user hardware. In it also includes some basic control functions such as button, LED. See Table 7 for details.

Table 7. Peripheral configuration functions

Function	Description
nvic_config	Configure interrupt priority
tmr_pwm_init	Configure PWM output-related timer, crm clock, GPIO and DMA
gpio_hall_init	Configure Hall sensor GPIOs (for BLDC use only)
tmr_hall_init	Configure Hall sensor timer and crm clocks (for BLDC use only)
tmr_sensorless_change_phase_init	Configure phase change timer and crm clock in sensorless mode (for sensorless BLDC)
hall_timer_init	Configure Hall sensor timer, crm clock and GPIO (for FOC use only)
encoder_time_init	Configure incremental photoelectric encoder timer, crm clock, GPIO and EXINT (for FOC only)
encoder_capture_timer_init	Photoelectric Encoder capture timer, crm clock, GPIO (for FOC/MT_METHOD only)
adc_ordinary_config	Configure ADC, DMA and GPIO of ADC ordinary channels
adc_preempt_config	Configure ADC, DMA and GPIO of ADC preempted channels
speed_timer_init	Configure speed control clock (tmr) and crm clock
uart_init	Configure UART and UART-related crm clock and GPIO
button_switch_init	Switch button GPIO and crm clock configuration (for BLDC hall sensor only)
button_exint_init	Button interrupt event EXINT configuration
led_config	LED GPIO and crm clock initialization configuration
led_on	LED ON
led_off	LED OFF
led_toggle	LED toggle (from on to off, or from off to on)
led_init	LED initialization configuration
led_blink	LED flashes
mode_switch_init	Switch button configuration
current_offset_tmr_setting	Current offset timer configuration (for BLDC only)
bldc_angle_init_config	TMR configuration of initial angle detection (for sensorless BLDC only)

tmr_sensorless_change_phase_init	TMR configuration for sensorless phase change (for sensorless BLDC only)
tmr_read_emf_init	TMR configuration for sensorless zero-crossing detection in continuous sampling mode (for sensorless BLDC only)
bldc_sensorless_detectEMF_config	TMR, ADC, and DMA configuration for sensorless zero-crossing detection (for sensorless BLDC only)
foc_angle_init_config	TMR and ADC configuration for initial angle detection (FOC sensorless only)
motor_parameter_ID_config	Set TMR and ADC relating to motor winding parameter identification
get_int_vref_cal_ratio	Detect VDDA reference voltage ratio, used for ADC value calibration
hwdiv_config	Hardware divider configuration (for AT32L021 series only)
opa_init	Busbar Current Amplification OP Configuration (AT32M412 and other chips with internal OP only)
opa_calibration	Busbar Current Amplification OP Calibration (AT32M412 and other chips with internal OP only)
ocp_cmp_config	Busbar Current Protection CMP Configuration (AT32M412 and other chips with internal CMP only)
cmp2_config	Back-EMF Internal CMP Configuration (AT32M412 and other chips with internal CMP only)
tmr_comp_capture_init	TMR Capture Channel Configuration for Back-EMF Internal Comparator (AT32M412 and other chips with internal CMP only)
tmr_blank_trigger_init	TMR Configuration for Internal Comparator Blanking Trigger Source (AT32M412 and other chips with internal CMP only)
tmr_blank_init	TMR Configuration for Internal Comparator Blanking Signal (AT32M412 and other chips with internal CMP only)

4) mc_isr.c file

- This file contains interrupt functions, as shown in Table 8.

Table 8. Motor control interrupt functions

Function	Description
ADVTMR_PWM_CYCLE_IRQ	Control loop interrupt (PWM update interrupt)
ADVTMR_PWM_BRK_IRQ	Brake input interrupt (PWM output disable)
ADC_SHUNT_SAMP_READY_IRQ	Current/BEMF sensing complete interrupt
ENCODER_CAPTURE_IRQ	Encoder capture interrupt (for FOC only)
EXINT_ENCODER_IDX_IRQ	Encoder zero position interrupt (for FOC only)
HALL_CAPTURE_IRQ	Hall signal input capture interrupt
SPEED_LOOP_TIMER_IRQ	Speed loop interrupt (for FOC only)
SysTick_Handler	System interrupt (1ms), state machine program

BUTTON_EXINT_IRQ	External signals to activate the HALL self-learning interrupt function (for hall sensor) and button function
CHANGE_PHASE_IRQ	Sensorless phase change interrupt (for BLDC only)
READ_EMF_IRQ	Sensorless zero-crossing detection interrupt in continuous sampling mode (for BLDC continuous sampling mode)
ADVTMR_CH1_EMF_PULL_IRQ	The interrupt function for zero-crossing point detection with ADC in sensorless mode (ARTERY's proprietary pull-up circuit)

5) mc_type.h file

- This file holds global enumeration/type definitions, as shown in Table 9.

Table 9. List of motor control library enumeration

Enumeration/Type	Description
BLDC/FOC general parameters	
long_words_union	Singed number 16-bit and 32-bit integer variable union declaration
float_u32_union	Floating-point number and unsigned number 32-bit integer variable union declaration
firmware_id_type	Firmware ID type used to identify motor control mode
motor_control_mode	Motor control mode (open loop control, voltage control, Id tune mode, Iq tune mode, speed control, torque control, position control, etc.)
ctrl_source_type	Control source (software control, external circuit control)
process_state_type	Processing state
esc_state_type	Motor control process state machine (see Section 5.1.15.1)
err_code_type	Motor error type (over-voltage, under-voltage, over-temperature, over-current, encoder error, Hall error, winding parameter identification failure, hall self-learning failure)
shunt_nbr_type	Shunt current detection mode (1-shunt, 2-shunt or 3-shunt)
hall_learn_process_type	Hall self-learning process state
curr_offs_type	ADC current sampling offset value
current_type	Current variables
pid_ctrl_type	PID control loop variables
speed_type	Speed variables
value_type	Value type
hall_sensor_type	Hall sensor variables
ramp_cmd_type	Ramp command type
usart_data_index	UART queue variables
moving_average_type	Moving average variables
lowpass_filter_type	Low-pass filter type
usart_config_type	UART peripheral configuration type
ui_wave_param_type	UI wave display parameters
i_auto_tune_type	Current PI controller auto-tuning parameters
motor_param_id_type	Winding parameters identification
hall_learn_type	Hall self-learning variable
FOC parameters	
low_spd_ctrl_type	Low-speed voltage control type
encoder_calibrate_process_type	Encoder angle calibration process state
qd_type	d,q type

abc_type	a,b,c type
alphabeta_type	α,β type
i_dc_type	Bus current type
trig_components_type	Trigonometric function type
voltage_type	Voltage type
adc_trigger_type	ADC trigger type
pwm_duty_type	PWM duty type
pid_ctrl_dc_type	PID control loop type
rotor_angle_type	Rotor angle type
encoder_type	Encoder type
open_loop_type	Open loop type
position_type	Position type
angle_type	Angle type
field_weakening_type	Field weakening type
motor_volt_type	Voltage output type
motor_emf_type	BEMF detection type
state_observer_type	Sensorless observer type
sensorless_startup_type	Sensorless startup type
foc_angle_init_type	Initial angle detection type
rds_cali_type	MOS's $R_{DS(on)}$ calibration type
curr_decoupling_type	Current decouple control type
mtpa_type	MTPA type
three_phase_adc_channel	Three-Phase ADC channel type
BLDC parameters	
angle_init_type	Initial angle detection type (for sensorless BLDC only)
start_state_type	State machine for initial angle detection (for sensorless BLDC only)
calibrate_process_type	PWM input control neutral calibration process type
switch_mode_type	Forward-reverse mode /forward-brake mode /forward-reverse-brake mode
init_current_type	Current type for initial angle detection (for sensorless BLDC only)
angle_init_type	Initial angle detection type (for sensorless BLDC only)
i_bus_type	BUS current type
olc_type	Open loop type
emf_sample_type	BEMF sampling type (for sensorless BLDC only)
adc_sample_type	ADC sampling type
pwm_in_type	PWM input capture type
sound_type	Warning tone type
blank_type	Internal comparator blanking type

blank_trigger_type	Internal comparator blanking trigger source type

4 Motor control library functions

There are a six-step square wave motor control library (mc_bldc_kernal.lib) and a vector control motor control library (mc_foc_kernal.lib) available for uses to choose from according to their needs. All these two motor control libraries contain sensored and sensorless control functions. Yet it is worth noting that there is a need to perform software initialization settings by calling initialization functions before the use of motor control library. How to use motor control functions is detailed in the subsequent sections.

4.1 General-purpose motor control library functions

Initialization functions (they must be configured prior to motor control library use)

- **mc_param_init**

Table 10. mc_param_init

Name	Description
Function name	mc_param_init
Function prototype	flag_status mc_param_init(uint8_t fw_id);
Function description	Set initial parameters for motor control library
Input parameter	fw_id: control mode ID
Output parameter	NA
Return value	Set (successful) or Reset (failed)
Required preconditions	Get the fw_id by executing the get_fw_id
Called function	NA

Example

```
param_initial_rdy = mc_param_init(firmware_id);
```

Mathematical operation functions

- **atan2_fixed**

Table 11. atan2_fixed

Name	Description
Function name	atan2_fixed
Function prototype	int16_t atan2_fixed(int32_t y, int32_t x);
Function description	Fixed point arctan function
Input parameter 1	Y: y-axis signal
Input parameter 2	X: x-axis signal
Output parameter	NA
Return value	Signals after going through arctan function
Required preconditions	NA
Called function	NA

Example

```
local_degree = atan2_fixed(local_I.beta, local_I.alpha);
```

- **sqrt_fixed**

Table 12. sqrt_fixed

Name	Description
Function name	sqrt_fixed
Function prototype	void sqrt_fixed(int32_t input, int16_t *out_ptr);
Function description	Fixed-point square foot function
Input parameter	input: input parameter
Output parameter	out_ptr: output parameter
Return value	NA
Required preconditions	NA
Called function	NA

Example

```
local_degree = sqrt_fixed(cal_temp, &cal_temp1);
```

Motor winding parameter auto identification

- **param_identify**

Table 13. param_identify

Name	Description
Function name	param_identify
Function prototype	void param_identify(motor_param_id_type *mot_param_id_handler);
Function description	Motor winding parameter auto identification
Input parameter	mot_param_id_handler: motor_param_id_type pointer
Output parameter	mot_param_id_handler: motor_param_id_type pointer
Return value	NA
Required preconditions	NA
Called function 1	tmr_channel_enable
Called function 2	tmr_channel_value_set

Example

```
param_identify(&motor_param_ident);
```

- **param_id_process**

Table 14. param_id_process

Name	Description
Function name	param_id_process
Function prototype	void param_id_process(motor_param_id_type *mot_param_id_handler);
Function description	Motor winding parameter identification process
Input parameter	mot_param_id_handler: motor_param_id_type pointer
Output parameter	mot_param_id_handler: motor_param_id_type pointer
Return value	NA
Required preconditions	NA
Called function	tmr_channel_value_set

Example

```
param_id_process(&motor_param_ident);
```

4.2 Motor control library functions in vector control mode

Current sampling functions

- current_read_foc_1shunt

Table 15. current_read_foc_1shunt

Name	Description
Function name	current_read_foc_1shunt
Function prototype	void current_read_foc_1shunt(current_type *curr_handler, uint8_t sector_handler);
Function description	Read bus current value in vector control mode to re-build a 3-shunt current
Input parameter 1	curr_handler: point to the bus current offset value in the structure current_type
Input parameter 2	sector_handler: point to the voltage control area in the voltage_type
Output parameter	curr_handler: point to the 3-shunt current value in the structure current_type
Return value	NA
Required preconditions	NA
Called function	adc_preempt_conversion_data_get

Example

```
current_read_foc_1shunt(&current, volt_cmd.sector);
```

- rds_auto_calibration

Table 16. rds_auto_calibration

Name	Description
Function name	rds_auto_calibration
Function prototype	void rds_auto_calibration(rds_cali_type *rds_cali_handler, current_type *curr_handler, voltage_type *volt_handler);
Function description	MOS's $R_{DS(on)}$ auto calibration in E_BIKE_SCOOTER mode
Input parameter 1	rds_cali_handler: point to the d/q-axis current filter value in the structure rds_cali_type
Input parameter 2	curr_handler: point to the bus current filter value in the structure current_type
Input parameter 3	volt_handler: point to the d/a-axis voltage in the structure voltage_type
Output parameter	rds_cali_handler: point to the MOS's $R_{DS(on)}$ auto calibration in the structure rds_cali_type
Return value	NA
Required preconditions	NA
Called function	lowpass_filtering

Example

```
rds_auto_calibration(&Rds_Cali, &current, &volt_cmd);
```

- select_two_adc_channels

Table 17. select_two_adc_channels

Name	Description
Function name	select_two_adc_channels
Function prototype	void select_two_adc_channels(void);*curr_handler, voltage_type *volt_handler);
Function description	Dual ADC Current Sampling Channel Switch (TWO_ADC_CONVERTERS && THREE_SHUNT only)
Input parameter 1	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called function	adc_preempt_channel_set, tmr_channel_value_set

Example

```
select_two_adc_channels();
```

- current_read_foc_3shunt_2adc

Table 18. current_read_foc_3shunt_2adc

Name	Description
Function name	current_read_foc_3shunt_2adc
Function prototype	void current_read_foc_3shunt_2adc(current_type *curr_handler);
Function description	Reading Dual ADC Channel Values and Calculating Three-phase Current Values during Vector Control (TWO_ADC_CONVERTERS && THREE_SHUNT only)
Input parameter 1	curr_handler：Pointer to three-phase current offset values of the current_type structure
Output parameter	curr_handler：Pointer to three-phase current values of the current_type structure
Return value	NA
Required preconditions	NA
Called function	adc_preempt_conversion_data_get

Example

```
current_read_foc_3shunt_2adc(&current);
```

Hall sensor functions

- hall_delta_theta_calculation

Table 19. hall_delta_theta_calculation

Name	Description
Function name	hall_delta_theta_calculation
Function prototype	int16_t hall_delta_theta_calculation(speed_type *rotor_speed_handler, rotor_angle_type *rotor_angle_handler, hall_sensor_type *hall_handler);
Function description	Get Hall senor rotor angle variation
Input parameter 1	hall_handler: point to HALL state in the structure hall_sensor_type
Input parameter 2	rotor_angle_handler: point to the rotor angle in the structure rotor_angle_type
Input parameter 3	rotor_speed_handler: point to the rotor speed in the structure speed_type
Output parameter	NA
Return value	Rotor angle variation
Required preconditions	NA
Called function	NA

Example

```
hall.theta_inc = hall_delta_theta_calculation(&rotor_speed_hall, &rotor_angle_hall, &hall);
```

PWM function

- svpwm_3shunt

Table 20. svpwm_3shunt

Name	Description
Function name	svpwm_3shunt
Function prototype	void svpwm_3shunt(voltage_type *volt_handler, pwm_duty_type *pwm_duty_handler);
Function description	Space vector pulse width modulation function (for 3-shunt use)
Input parameter 1	volt_handler: point to the parameters of the structure voltage_type
Output parameter	pwm_duty_handler: point to the parameters of the structure pwm_duty_type
Return value	NA
Required preconditions	NA
Called function	NA

Example

```
pwm_duty = svpwm_3shunt(&volt_cmd, &pwm_duty);
```

- svpwm_2shunt

Table 21. svpwm_2shunt

Name	Description
Function name	svpwm_2shunt
Function prototype	void svpwm_2shunt(voltage_type *volt_handler, pwm_duty_type *pwm_duty_handler);
Function description	Space vector pulse width modulation function (for 2-shunt use)
Input parameter 1	volt_handler: point to the parameters of the structure voltage_type
Output parameter	pwm_duty_handler: point to the parameters of the structure pwm_duty_type
Return value	NA
Required preconditions	NA
Called function	NA

Example

```
pwm_duty = svpwm_2shunt(&volt_cmd, &pwm_duty);
```

- svpwm_1shunt

Table 22. svpwm_1shunt

Name	Description
Function name	svpwm_1shunt
Function prototype	void svpwm_1shunt(voltage_type *volt_handler, pwm_duty_type *pwm_duty_handler);
Function description	Space vector pulse width modulation function (for 1-shunt use)
Input parameter 1	volt_handler: point to the parameters of the structure voltage_type
Output parameter	pwm_duty_handler: point to the parameters of the structure pwm_duty_type
Return value	NA
Required preconditions	NA
Called function	pwm_shift

Example

```
pwm_duty = svpwm_1shunt(&volt_cmd, &pwm_duty);
```

- pwm_shift

Table 23. pwm_shift

Name	Description
Function name	pwm_shift
Function prototype	adc_trigger_type pwm_shift(int16_t *dTa, int16_t *dTb, int16_t *dTc, int16_t Ta, int16_t Tb, int16_t Tc, pwm_duty_type *pwm_duty_handler);
Function description	Pulse width modulation shift control (for 1-shunt use)
Input parameter 1	Ta: max. 3-phase duty cycle
Input parameter 2	Tb: second-largest 3-phase duty cycle
Input parameter 3	Tc: min. 3-phase duty cycle
Input parameter 4	pwm_duty_handler: pointer to the parameters of the structure pwm_duty_type
Output parameter 1	dTa: offset value for the maximum 3-phase duty cycle
Output parameter 2	dTb: offset value for the second-largest 3-phase duty cycle
Output parameter 3	dTc: offset value for the minimum 3-phase duty cycle
Return value	adc_trigger_type: current sampling time point
Required preconditions	NA
Called function	NA

Example

```
adc_trig = pwm_shift(&dTa, &dTb, &dTc, Ta, Tb, Tc, &local_pwm_duty);
```

Sensorless vector control functions

- startup_alpha_axis

Table 24. startup_alpha_axis

Name	Description
Function name	startup_alpha_axis
Function prototype	flag_status startup_alpha_axis(sensorless_startup_type *startup_handler, qd_type *Iqdref_handler);
Function description	Align and go startup function
Input parameter 1	Iqdref_handler: pointer to the parameters of the structure qd_type
Input parameter 2	startup_handler: pointer to the parameters of the structure sensorless_startup_type
Output parameter	NA
Return value	flag_status: return SET (successful) or RESET (failed)
Required preconditions	NA
Called function	NA

Example

```
startup.closeloop_rdy = startup_alpha_axis(&startup, &Iref);
```

- flag_status startup_angle_init

Table 25. flag_status startup_angle_init

Name	Description
Function name	flag_status startup_angle_init
Function prototype	flag_status startup_angle_init(sensorless_startup_type *startup_handler, qd_type *Iqdref_handler);
Function description	Align and go startup function with initial angle
Input parameter 1	Iqdref_handler: pointer to the parameters of the structure qd_type
Input parameter 2	startup_handler: pointer to the parameters of the structure sensorless_startup_type
Output parameter	NA
Return value	flag_status: return SET (successful) or RESET (failed)
Required preconditions	NA
Called function	NA

Example

```
startup.closeloop_rdy = startup_angle_init(&startup, &Iref);
```

- foc_sensorless_angle_init

Table 26. foc_sensorless_angle_init

Name	Description
Function name	foc_sensorless_angle_init
Function prototype	void foc_sensorless_angle_init(foc_angle_init_type *angle_detect_handler, current_type *curr_handler);
Function description	Initial angle detection function
Input parameter 1	angle_detect_handler: pointer to the parameters of the structure foc_angle_init_type
Input parameter 2	curr_handler: pointer to the parameters of the structure current_type
Output parameter	angle_detect_handler: pointer to the motor initial angel of the structure foc_angle_init_type
Return value	NA
Required preconditions	NA
Called function 1	tmr_output_enable
Called function 2	tmr_channel_value_set
Called function 3	tmr_counter_enable

Example

```
foc_sensorless_angle_init(&angle_detector, &current);
```

- obs_pll_execute

Table 27. obs_pll_execute

Name	Description
Function name	obs_pll_execute
Function prototype	int16_t obs_pll_execute(state_observer_type *state_obs_handler, int16_t hBemf_alfa_est, int16_t hBemf_beta_est);
Function description	Quadrature-component-based PLL function
Input parameter 1	state_obs_handler: pointer to the parameters of the structure state_observer_type
Input parameter 2	hBemf_alfa_est: the estimated BEMF α-axis voltage
Input parameter 3	hBemf_beta_est: the estimated BEMF β-axis voltage
Output parameter	NA
Return value	Rotor speed
Required preconditions	NA
Called function 1	arm_sin_q15
Called function 2	arm_cos_q15
Called function 3	pi_controller

Example

```
hRotor_Speed = obs_pll_execute(state_obs_handler, state_obs_handler->hBemf_alpha_est,
state_obs_handler->hBemf_beta_est);
```

- rotor_angle_sensorless

Table 28. rotor_angle_sensorless

Name	Description
Function name	rotor_angle_sensorless
Function prototype	int16_t rotor_angle_sensorless(state_observer_type *state_obs_handler, motor_volt_type *motor_volt_handler);
Function description	Rotor angle observer
Input parameter 1	state_obs_handler: pointer to the parameters of the structure state_observer_type
Input parameter 2	motor_volt_handler: pointer to the parameters of the structure motor_volt_type
Output parameter	NA
Return value	Rotor angle
Required preconditions	Executing motor_volt_calc or motor_volt_read returns driver output α-axis and β-axis voltage
Called function	obs_pll_execute

Example

```
state_observer.elec_angle = rotor_angle_sensorless(&state_observer, &motor_voltage);
```

4.3 Motor control library functions in 6-step square wave mode

6-step square wave functions

- calc_adc_sample_point

Table 29. calc_adc_sample_point

Name	Description
Function name	calc_adc_sample_point
Function prototype	void calc_adc_sample_point (adc_sample_type *adc_sample,int16_t pwm_duty);
Function description	ADC sample point calculation function (including current sample point and BEMF sample point, see note [7])
Input parameter 1	adc_sample: point to the parameters of the structure adc_sample_type
Input parameter 2	pwm_duty: current PWM duty cycle
Output parameter	adc_sample: point to the adc_sample_point of the structure adc_sample_type
Return value	NA
Required preconditions	NA
Called function	NA

Example

```
adc_sample_point_set(&adc_sample, pwm_comp_value);
```

Note [7]: BEMF sample point is only applicable to sensorless mode. In sensored mode, only current sample point is calculated.

6-step square wave sensorless functions

- bldc_sensorless_angle_init

Table 30. bldc_sensorless_angle_init

Name	Description
Function name	bldc_sensorless_angle_init
Function prototype	void bldc_sensorless_angle_init(angle_init_type *angle_init);
Function description	Bus current sampling function (for motor initial angle detection)
Input parameter 1	angle_init: point to the parameters of the structure angle_init_type
Output parameter	angle_init: point to the current of the structure angle_init_type
Return value	NA
Required preconditions	NA
Called function 1	tmr_channel_value_set
Called function 2	disable_mosfet
Called function 3	tmr_output_enable
Called function 4	tmr_counter_enable

Example

```
bldc_sensorless_angle_init(PWM_ADVANCE_TIMER,&angle_init,angle_init_step);
```

- angle_init_estimation

Table 31. angle_init_estimation

Name	Description
Function name	angle_init_estimation
Function prototype	uint8_t angle_init_estimation(angle_init_type *angle_init);
Function description	Initial angle estimation
Input parameter 1	angle_init: pointer to the parameters of the structure angle_init_type
Input parameter 2	NA
Output parameter	angle_init: pointer to the parameters of the structure angle_init_type
Return value	Return six-step square wave control phase (from 1 to 6) corresponding to the detected initial angle
Required preconditions	bldc_sensorless_angle_init
Called function	NA

Example

```
hall.state = angle_init_estimation(&angle_init);
```

5 Motor control library application example structure

5.1 State machine overview

The state machine flow chart is shown in Figure 9. It includes such state as Idle, Safety ready, Angle init, Starting, Running, Free run, I_tune, Enc_align and Error.

5.1.1 State descriptions

Idle*

The initial state of a state machine. Motor stays idle in this state. And the state machine returns to “Idle” state when a failure is cleared or motor stops running.

Safety ready*

This state indicates that the motor can start up safely as all parameters are already set and the current offset value is available.

Angle init

This refers to the state for initial angle detection prior to motor startup in sensorless control mode. The procedure jumps to “Starting” state as long as the initial angle is obtained. Note that “Starting state” is applicable to the sensorless control mode only. If there is no need to detect initial angle, the “Starting state” can be skipped.

Starting

In sensorless control mode, after the initial angle is detected, the procedure jumps to “Starting state” in which the user can set driver parameters, peripherals and conditions to switch back to closed loop.

Running*

This state indicates that the motor is running, and the user can adjust parameters (target speed, target current) or stop motor via UI interface.

Free run

This state is similar to Stop mode. In this state, the driver stops output, and the motor speed will slowly reduce to zero. This state is maintained until the motor stops running. After a full stop of motor, the state machine then returns to “Safety ready” state.

I_tune*

This state is used to tune PID controller parameters. In this state, the user can tune the target current, current loop KP and KI values via UI software to generate a step current.

Enc_align

This represents the zero-position calibration state of an encoder. With the UI interface, the user can enable zero-position calibration feature. Typically, after a driver reset, it is necessary to activate zero-position calibration. If no such action implemented, an auto zero-position calibration will be triggered before motor startup. This state is skipped for a motor without encoder.

Winding param Id

This refer to the state of motor winding parameter identification. The user can enable this feature via UI interface. The winding parameter identification feature is often used for tuning a new motor. The tuned parameters can be used for sensorless vector control and current PI controller parameters self-tuning.

Auto learn

This refers to a hall state sequence self-learning feature in our motor control library. It can be used in six-step square wave mode and vector control mode.

Error*

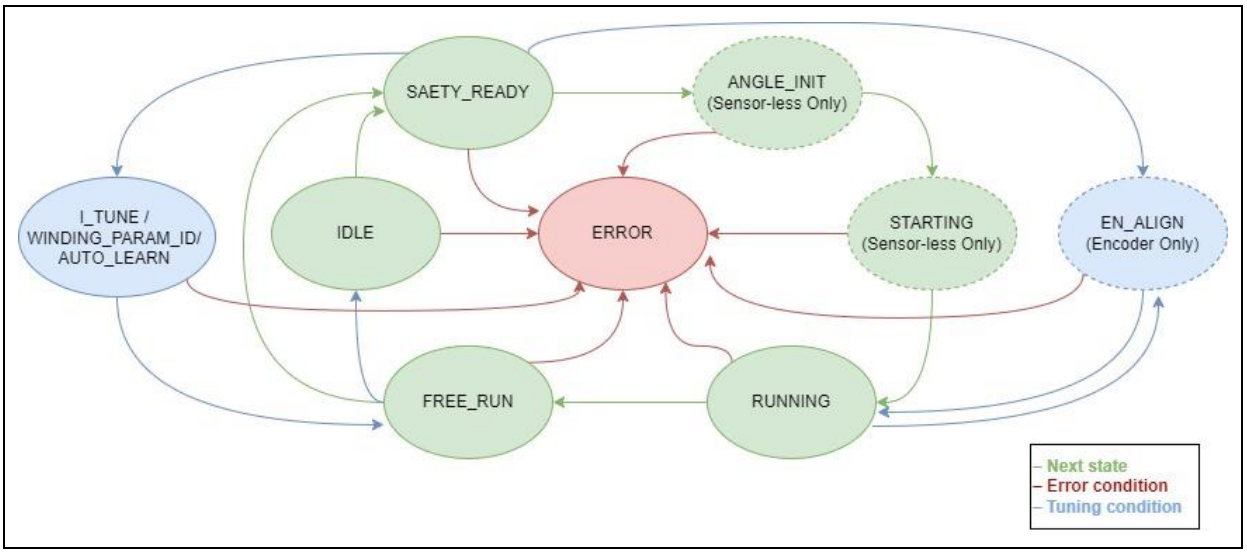
The state machine jumps to “Error” state in case of any failures.

Note [8]: The states marked with “” are executed continuously until other event (user operation or any fault) is generated.*

5.1.2 State machine procedure

Figure 9 illustrates the flow chart of the state machine. “Green” circles represent a state machine switch process in normal conditions, while “Blue” circles represent a switch process under special circumstances. To adjust the current PID control parameters, it can be switched to the I_TUNE state. To use motor winding parameter identification function, jump to the WINDING_PARAM_ID. To use HALL self-learning feature, jump to the AUTO_LEARN. If an encoder is to be used, it is necessary to switch to the ENC_ALIGN state to calibrate encoder before startup. The “Red” one indicates the system error. The state machine jumps to the ERROR state if any error (such as over-voltage, over-current, etc.) occurs during program execution.

Figure 9. State machine process flow



6 Revision history

Table 32. Document revision history

Date	Version	Revision note
2022.11.18	2.0.0	Initial release.
2022.11.18	2.0.1	Updated some descriptions and terms
2022.11.24	2.0.2	Updated some descriptions and terms
2022.12.01	2.0.3	Updated code definitions and descriptions
2022.12.10	2.0.4	Updated motor library codes
2022.12.23	2.0.5	Revised document formats.
2022.12.29	2.0.6	Revised text errors
2023.03.02	2.0.7	Added Keil V5.33 compiling limitation, software requirements, united function description formats, code descriptions.
2023.04.20	2.0.8	Added macro definition of positioning control, interrupt functions, control functions and relevant descriptions.
2023.10.05	2.1.0	Updated the contents relating to motor control program architecture, functions and files.
2024.02.27	2.1.1	Added the following descriptions: motor winding parameter identification, current PI controller parameter tuning, hall self-learning macro definition, interrupt functions, control functions, state machine.
2024.05.24	2.1.2	Updated code definitions and descriptions.
2024.12.27	2.1.3	Added applicable product series, updated some definitions and description, added IAR system and the corresponding Flash ROM configuration
2025.03.28	2.1.4	Added upwind/downwind startup and MTPA/MTPV macro definitions description
2025.10.09	2.1.5	Added new supported models, fixed flash configuration table, parameter updates, modified function descriptions.

IMPORTANT NOTICE – PLEASE READ CAREFULLY

Purchasers are solely responsible for the selection and use of ARTERY's products and services; ARTERY assumes no liability for purchasers' selection or use of the products and the relevant services.

No license, express or implied, to any intellectual property right is granted by ARTERY herein regardless of the existence of any previous representation in any forms. If any part of this document involves third party's products or services, it does NOT imply that ARTERY authorizes the use of the third party's products or services, or permits any of the intellectual property, or guarantees any uses of the third party's products or services or intellectual property in any way.

Except as provided in ARTERY's terms and conditions of sale for such products, ARTERY disclaims any express or implied warranty, relating to use and/or sale of the products, including but not restricted to liability or warranties relating to merchantability, fitness for a particular purpose (based on the corresponding legal situation in any unjudicial districts), or infringement of any patent, copyright, or other intellectual property right.

ARTERY's products are not designed for the following purposes, and thus not intended for the following uses: (A) Applications that have specific requirements on safety, for example: life-support applications, active implant devices, or systems that have specific requirements on product function safety; (B) Aviation applications; (C) Aerospace applications or environment; (D) Weapons, and/or (E) Other applications that may cause injuries, deaths or property damages. Since ARTERY products are not intended for the above-mentioned purposes, if purchasers apply ARTERY products to these purposes, purchasers are solely responsible for any consequences or risks caused, even if any written notice is sent to ARTERY by purchasers; in addition, purchasers are solely responsible for the compliance with all statutory and regulatory requirements regarding these uses.

Any inconsistency of the sold ARTERY products with the statement and/or technical features specification described in this document will immediately cause the invalidity of any warranty granted by ARTERY products or services stated in this document by ARTERY, and ARTERY disclaims any responsibility in any form.